

Asynchronous JavaScript Mastery Exercises

Phase 1: Callback Foundations (Week 1)

Exercise 1.1: Basic Callback Timer

Create a function `delayedGreeting(name, delay, callback)` that waits for the specified delay (in ms) then calls the callback with a greeting message.

Goal: Understand callback execution flow

Exercise 1.2: Callback Hell Recreation

Simulate reading three files sequentially using `setTimeout`. Each file read takes 1 second and depends on data from the previous one.

Goal: Experience why callback hell is problematic

Exercise 1.3: Error-First Callbacks

Build a function `fetchUserData(userId, callback)` that randomly succeeds or fails, using Node.js style error-first callbacks: `callback(error, data)`

Goal: Learn proper error handling patterns

Phase 2: Promise Fundamentals (Week 2)

Exercise 2.1: Convert Callbacks to Promises

Take your Exercise 1.2 code and refactor it using Promises with `.then()` and `.catch()`

Goal: See how Promises improve readability

Exercise 2.2: Promise.all() Race

Fetch data from these 3 free APIs simultaneously:

- <https://jsonplaceholder.typicode.com/users/1>
- <https://jsonplaceholder.typicode.com/posts/1>
- <https://jsonplaceholder.typicode.com/comments/1>

Display all results only when ALL requests complete.

Goal: Master parallel async operations

Exercise 2.3: Promise.race() Timeout

Implement a `fetchWithTimeout(url, timeout)` function that races between a fetch request and a timeout. If the request takes longer than the timeout, reject with "Request timed out".

Goal: Understand Promise.race() practical use

Exercise 2.4: Build Your Own Promise

Create a simple Promise implementation from scratch that supports `.then()` and `.catch()`. This is advanced but incredibly educational.

Goal: Deep understanding of how Promises work internally

Phase 3: Async/Await Mastery (Week 3)

Exercise 3.1: Refactor to Async/Await

Convert all your Exercise 2.1 code to use async/await syntax with try/catch blocks.

Goal: Master modern async syntax

Exercise 3.2: Sequential vs Parallel

Fetch posts from JSONPlaceholder (posts 1-5). First do it sequentially (one after another), then optimize to run in parallel. Compare execution times using `console.time()`.

Goal: Understand performance implications

Exercise 3.3: Error Recovery

Create an async function that tries to fetch data from multiple API endpoints. If one fails, try the next one. Only fail if all attempts fail.

Goal: Advanced error handling patterns

Exercise 3.4: Async Iteration

Write an async generator function that fetches paginated data from an API. Use `for await...of` to consume it.

Goal: Learn advanced async patterns

Phase 4: HTTP Methods Practice (Week 4)

Exercise 4.1: REST API Client

Build a complete CRUD interface for JSONPlaceholder todos:

- GET all todos
- GET single todo
- POST new todo
- PUT update todo
- DELETE todo

Use `fetch()` with proper HTTP methods.

Goal: Master all HTTP verbs

Exercise 4.2: Build a Mini Backend

Using Node.js and Express, create an API with these endpoints:

- GET `/users` - return user list
- POST `/users` - create new user
- PUT `/users/:id` - update user
- DELETE `/users/:id` - delete user

Store data in memory (array).

Goal: Understand both sides of HTTP

Exercise 4.3: Request Interceptors

Create a wrapper around `fetch()` that:

- Adds authentication headers to all requests
- Logs all requests and responses
- Retries failed requests up to 3 times
- Handles rate limiting (429 status)

Goal: Production-ready HTTP handling

Phase 5: Real-World Projects (Week 5-6)

Project 1: Weather Dashboard

Requirements:

- Fetch weather from OpenWeatherMap API
- Search for multiple cities
- Display data with loading states
- Handle errors gracefully
- Cache results to avoid duplicate API calls

Skills: API integration, state management, error handling

Project 2: Image Gallery with Infinite Scroll

Requirements:

- Use Unsplash API
- Load 10 images initially
- Load more when scrolling to bottom
- Show loading spinner
- Handle network errors

Skills: Pagination, scroll events, async coordination

Project 3: Real-Time Chat Simulator

Requirements:

- Simulate WebSocket behavior with long polling
- Send and receive messages
- Show typing indicators
- Handle connection drops
- Retry failed message sends

Skills: Advanced async patterns, state management

Project 4: File Upload with Progress

Requirements:

- Upload files to a mock endpoint
- Show upload progress
- Support multiple simultaneous uploads
- Handle failures and retries
- Cancel uploads mid-flight

Skills: Streams, AbortController, concurrent operations

Phase 6: Advanced Challenges (Ongoing)

Challenge 1: Race Condition Debugger

Create scenarios with race conditions, then fix them using proper async patterns.

Challenge 2: Custom Retry Logic

Build a generic `retryAsync(fn, options)` utility that retries any async function with exponential backoff.

Challenge 3: Request Queue

Implement a queue that limits concurrent API requests (max 3 at a time) while queuing others.

Challenge 4: Async Middleware Pipeline

Build a middleware system like Express, where async functions can process requests in sequence.

Resources

Interactive Learning

- **JavaScript.info** - Promises section: <https://javascript.info/promise-basics>
- **Async/Await**: <https://javascript.info/async-await>
- **freeCodeCamp** - APIs and Microservices: <https://www.freecodecamp.org/learn>

Video Courses

- **Traversy Media** - Async JavaScript Crash Course (YouTube)
- **Web Dev Simplified** - JavaScript Promises & Async/Await (YouTube)
- **The Net Ninja** - Asynchronous JavaScript Tutorial (YouTube playlist)

Practice Platforms

- **Exercism.org** - JavaScript Track (async exercises)
- **LeetCode** - JavaScript async problems
- **Codewars** - Filter for async JavaScript katas

Documentation

- **MDN Web Docs** - Promise: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
- **MDN** - async/await: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
- **MDN** - Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Books

- **"Eloquent JavaScript" Chapter 11** - Asynchronous Programming (free online)
- **"You Don't Know JS: Async & Performance"** by Kyle Simpson

Free APIs for Practice

- JSONPlaceholder: <https://jsonplaceholder.typicode.com/>
- OpenWeatherMap: <https://openweathermap.org/api> (free tier)
- REST Countries: <https://restcountries.com/>
- Dog API: <https://dog.ceo/dog-api/>
- Unsplash: <https://unsplash.com/developers>

Tools

- **Postman** - Test HTTP requests: <https://www.postman.com/>
- **JSON Server** - Mock REST API: <https://github.com/typicode/json-server>
- **RequestBin** - Inspect HTTP requests: <https://requestbin.com/>

Study Schedule Suggestion

Week 1: Exercises 1.1-1.3 + JavaScript.info Callbacks

Week 2: Exercises 2.1-2.4 + JavaScript.info Promises

Week 3: Exercises 3.1-3.4 + MDN async/await

Week 4: Exercises 4.1-4.3 + Build mini REST API

Week 5-6: Pick 2-3 projects from Phase 5

Ongoing: Advanced challenges as needed

Pro Tips

1. **Code every day:** Even 30 minutes of focused practice beats cramming
2. **Build without tutorials:** Try solving before looking at solutions
3. **Debug intentionally:** Break your code on purpose to understand errors
4. **Refactor relentlessly:** Rewrite the same exercise 3 different ways
5. **Teach others:** Explain concepts to solidify understanding
6. **Read error messages:** They're teaching you something specific

Next Step: Start with Exercise 1.1 today. Set a timer for 30 minutes and just code. Don't aim for perfection - aim for practice.